



Inputlog

Reengineering Tasks
Joeri Rammelaere (mail)

June 2014

GUI refactoring

Beschrijving

In het project **GUI**, voornamelijk in GUI/Tabs, zit tamelijk veel “functionaliteit” verstrengeld in de klassen die interageren met de GUI. Idealiter zou er een strikte opdeling zijn, klassen die met de GUI interageren en enkel waarden opvragen/weergeven, en klassen die deze parameters binnen krijgen en hiermee effectief iets uitvoeren. Dit zorgt er onder andere voor dat deze functionaliteit ge-unittest kan worden. De GUI maakt momenteel gebruik van WinForms, het is echter niet ondenkbaar dat er ooit richting WPF of een ander, flexibeler framework geëvolueerd wordt. Deze evolutie zal uiteraard ook eenvoudiger zijn als de GUI interactie en functionaliteit netjes gescheiden zijn.

Uitgelicht

GUI/Tabs/Analyze/Analyze.cs en GUI/Tabs/Preprocess/Preprocess.cs zijn 2 voorbeelden van tamelijk monolithische klassen (+-1000 LOC)

Concreet

- GUI interactie en functionaliteit splitsen: maximaliseer Test Coverage
- Stel je de vraag “wat moet er gebeuren als ik deze zelfde functionaliteit op een andere plek (bv via een wizard) zou willen aanbieden?”
- De “application flow” documenteren: welke klassen worden aangeroepen (en in welke volgorde) door welke knoppen?

Progress reporting en error handling

Beschrijving

Een ander punt dat aandacht nodig heeft in het GUI-gedeelte is Progress/Error reporting bij analyses. De groene balk onderaan dit tabblad is momenteel veelal helemaal *leeg* of *vol*, wat niet echt de bedoeling is. Qua error reporting is het niet helemaal duidelijk wat er gebeurt wanneer er exceptions opduiken in de verschillende analyses: wanneer wordt er verder gegaan met het volgende bestand of de volgende analyse, wanneer wordt er gerapporteerd, wat als de fout niet catastrofaal hoeft te zijn, ...

Om dit systeem te onderzoeken en/of uit te breiden, zal je ook de eigenlijke analyse-classes nodig hebben. Deze bevinden zich in het project `inputlog.core`, iedere analyse heeft een eigen map in `Core/Analyses`. Voor langere analyses zou bijvoorbeeld progress reporting tijdens de analyse ook nuttig kunnen zijn.

Uitgelicht

De Revisie-analyse (`InputLog.Core.Analyses.Revision.Notations.RevisionMatrixAnalysis`) is een tamelijk intensieve, foutgevoelige analyse die extra onder de loep genomen dient te worden

Concreet

- Huidige situatie qua progress reporting en error handling in kaart brengen
- Verbeteringen/herstellingen bespreken en uitwerken
- Mechanisme documenteren: waar moet rekening mee worden gehouden bij het toevoegen van nieuwe analyses?
- Uunittests voorzien in de mate van het mogelijke

IDFX en Analysis IO

Beschrijving

Het inlezen van IDFX en analyse .xml bestanden, en het uitschrijven van IDFX bestanden gebeurt in Core/IO. Voor het uitschrijven van analyse resultaten bevindt zich in iedere analyse-map in Core/Analyses een XMLWriter. Ook in Core/Events zijn ReadXml() en WriteXml() methodes voorzien die gebruikt worden bij het serializeren van AnalysisDescriptions voor processing op de Server.

Het is meteen duidelijk dat hier overlap van functionaliteit is, echter is het niet helemaal duidelijk of en wat de verschillen zijn. In Core/IO/Xml/XmlElements zijn Tags gedefinieerd voor het uitschrijven van analyses, deze worden in de meeste AnalysisXMLWriters opnieuw gedefinieerd (of gewoon als string ingevoerd zonder eerdere definitie, mijn excuses ...).

Voor sommige analyses is het inlezen bovendien nog niet voorzien. De XmlWriterFactory-structuur is uiterst flexibel maar dit brengt ook de nodige complexiteit met zich mee: dit kan mogelijk herbekeken worden. Ook het voorzien van een aparte reader voor ieder analysetype in Core/IO/AnalysisXML/Input is mijns inziens overbodig, mogelijk is dit met reflectie eenmalig op te lossen.

Tot slot is het nuttig om het outputformaat van de analyses te standardiseren. Analyses die worden uitgeschreven in het module-block-element formaat (zie Source, Fluency, ...) worden uitgeschreven kunnen automatisch horizontaal gemerged worden. De overige analyses hebben allemaal een andere structuur/tagnames.

Uitgelicht

In de functie Events() van InputLog.Core.IO.AnalysisXML.Input.BasicXmlReader worden momenteel alle mogelijke tags opgelijst die kunnen voorkomen in een general analysis. Voor de GeneralEyetrack en Linguistic analysis moet een soortgelijke functie voorzien worden die alle tags individueel inleest ...

Concreet

- Code duplicatie in uitschrijven en inlezen van XML wegwerken
- Output formaten standardiseren
- Input factories en structuur vereenvoudigen/automatiseren
- Unittests en documentatie voorzien

Kleinere dingen

Core Utils uitzoeken

De map Core/Util bevat allerlei functionaliteit . . . Het is echter niet duidelijk wát hier allemaal voorzien is en in hoeverre er overlap is. Deze klassen moeten uitgezocht worden en duidelijk gedocumenteerd in de reference guide.

Constantes naar settings verhuizen

Op meerdere plaatsen worden arbitraire waarden in constantes gedefinieerd, bijvoorbeeld de minimum time threshold is de Fluency analyse. Deze instellingen kunnen allemaal naar een “advanced” tabblad in de settings worden verhuisd (en gedocumenteerd).

Warnings wegwerken

Dit zijn er nu vermoedelijk zo’n 200 . . . waardoor je nooit de nuttige warnings opmerkt. In de mate van het mogelijke zouden deze weggewerkt moeten worden, waar nodig met het gebruik van Pragma’s ([link](#)).